CSC415-03 Operating System Principles
Spring 2026

File System Project

Created By:

Jiawei Lyu ID: 923809065 Central GitHub Repository: **jiaweilyu0929**

Leslie Raya ID: 921813630

Alexandra Borders ID: 913630008

Yeraldin Crespo ID: 923523819

Repository Link: https://github.com/jiaweilyu0929/csc415-group-project.git

Description:

This project implements a persistent, block-based file system in C on top of a Logical Block Addressing (LBA) interface. All disk operations are performed using fixed-size 512-byte blocks through the provided LBAread and LBAwrite functions. As a result, all higher-level file system operations must be translated into block-aligned reads and writes, requiring careful handling of offsets, buffering, and metadata consistency.

The system is composed of four tightly integrated subsystems: the Volume Control Block (VCB), bitmap-based free space management, a directory-based metadata system, and a buffered file I/O layer.

Volume Control Block (Superblock)

The Volume Control Block is stored at block 0 and serves as the central metadata structure for the entire file system. It defines the layout of the volume, including the total number of blocks, block size, location and size of the bitmap, and the starting location and size of the root directory.

A magic number is stored in the VCB to determine whether the volume has been previously formatted. During initialization, the system reads block 0 and checks this value to decide whether to mount an existing file system or perform a full format. This allows the system to maintain persistence across executions.

The VCB also maintains a count of free blocks, which is updated whenever blocks are allocated or freed, ensuring that the system can efficiently track available space without scanning the entire bitmap.

Free Space Management (Bitmap)

Free space is managed using a bitmap stored immediately after the VCB. Each bit corresponds to a single block on the volume, where a value of 0 indicates that the block is free and 1 indicates that it is allocated.

The bitmap is loaded into memory during initialization and maintained as a global structure for fast access. Allocation is performed by scanning for contiguous free bits, allowing the system to assign blocks efficiently for files and directories. Deallocation clears the corresponding bits and updates the free block count in the VCB.

Because the bitmap is written back to disk after every modification, the system ensures that free space information remains consistent even if the program exits unexpectedly.

Directory Structure and Metadata

Directories are implemented as arrays of fixed-size entries (fs_dirent_t) stored contiguously on disk. Each entry contains metadata including:

- File name (up to 255 characters)
- File type (regular file or directory)
- Allocation information (starting block and number of blocks)
- File size in bytes
- Timestamps for creation, modification, and access

Directories themselves are treated as files containing these entries. Each directory is allocated a fixed number of blocks, and entries are stored sequentially within those blocks.

Special entries ("." and "..") are initialized for each directory to support navigation. The root directory is initialized so that both entries point to itself, ensuring consistent behavior when navigating upward from the root.

Path resolution is handled through a combination of canonicalization and traversal. Input paths are normalized into absolute form and then resolved by reading directory blocks from disk and searching for matching entries. This ensures that all operations reflect the current state of the file system on disk rather than relying on cached data.

File Operations and Buffered I/O

File operations are implemented through a buffered I/O layer that provides an interface similar to standard POSIX file operations. A File Control Block (FCB) table is used to manage open files, storing information such as file position, buffer state, access mode, and file metadata.

The system distinguishes between files stored on the custom file system and those on the host system. For volume-based files, operations are performed using block-level access.

Reading from a file involves:

- Determining the correct block and offset based on the current file position
- Loading the block into a buffer using LBAread
- Copying only the requested bytes into the user buffer
- Ensuring that reads do not exceed the actual file size

Writing to a file requires handling partial block updates. Since the underlying storage system only supports full block writes, a read-modify-write strategy is used:

- The target block is first read into memory
- The relevant portion is updated with new data
- The entire block is written back using LBAwrite

File flags such as O_CREAT, O_TRUNC, and O_APPEND are supported and influence how files are opened and modified. Metadata updates, including file size and timestamps, are written back to the directory structure when a file is closed to ensure persistence.

Integration with Shell Interface

The file system is integrated with the provided fsshell driver, allowing users to interact with it using familiar commands such as ls, cd, touch, cp, and rm. These commands invoke the implemented file system functions, providing a complete environment for testing and validation.

As demonstrated in the execution screenshots(pages 17–29), the system supports directory navigation, file creation, file copying, and interaction between the custom file system and the host system. This confirms that both metadata management and data storage are functioning correctly.

Overall, the system demonstrates a complete implementation of a block-based file system, including persistent storage, hierarchical directories, efficient free space management, and buffered file access.

Approach:

Our approach to building the file system is based on a layered and dependency-driven design, where each major component will be implemented in a sequence that reflects how a real file system is constructed. We identified four core subsystems that must be developed: the Volume Control Block (VCB), free space management, directory structure, and buffered file I/O layer. Each of these components depends on the previous, so they will be implemented in phases.

Phase 1: Volume Initialization and Formatting

We will begin by implementing the volume initialization process in fsInit.c. The first step is to define a fs_superblock_tstructure that will be written directly to block 0 of the volume. This structure will store essential metadata, including a magic number, block size, total number of blocks, bitmap location, root directory location, and free block count.

During initialization, the system will attempt to mount an existing volume by reading block 0 using LBAread. If the stored magic number and version match expected values, the file system will be mounted by loading the metadata into memory. If not, the system will perform a full format.

Formatting will involve:

- Initializing a bitmap to track free space
- Marking reserved blocks (superblock, bitmap, root directory) as used
- Allocating a fixed number of blocks for the root directory
- Writing all initialized structures to disk using LBAwrite

The bitmap will be stored starting at block 1 and will represent each block with a single bit. This approach is chosen because it provides efficient space usage and allows for fast scanning when allocating contiguous blocks.

Phase 2: Directory Structure and Path Resolution

Once the volume is initialized, we will implement the directory system in mfs.c. Directories will be represented as arrays of fs_dirent_t structures stored contiguously on disk. Each entry will contain metadata such as file name, file type, size, starting block, block count, and timestamps.

A critical component of this phase is path resolution. We plan to implement this in two stages:

1. Path Canonicalization
   All input paths will be converted into absolute paths by resolving ".", "..", and redundant separators. This ensures consistency across all file system operations.
2. Directory Traversal (ParsePath)
   We will implement a ParsePath function that walks the directory hierarchy by reading directory blocks from disk using LBAread. At each level, the function will search for matching entries and descend into subdirectories until the target is found or determined to be invalid.

This design ensures that all directory operations always reflect the current on-disk state rather than relying on cached data.

Using this infrastructure, we will implement directory operations such as:

- fs_mkdir: Allocate blocks for a new directory, initialize "." and "..", and insert a new entry into the parent directory
- fs_rmdir: Verify that a directory is empty before removing it and freeing its allocated blocks
- fs_opendir, fs_readdir, fs_closedir: Support directory iteration by loading entries into memory and maintaining iteration state across calls

We will also maintain the current working directory as a global string and validate all directory changes using the path resolution system.

Phase 3: Buffered File I/O

In the final phase, we will implement file operations in b_io.c using a buffered I/O model. We will maintain a File Control Block (FCB) table to track open files, including their current position, buffer state, and metadata.

The b_open function will locate files within the directory structure using a helper function (e.g., mfs_volume_open). If the file exists, it will initialize an FCB with the file's metadata. If the file does not exist and O_CREAT is specified, a new directory entry will be created and space will be allocated using the bitmap.

For reading (b_read), we will:

- Calculate the correct block and offset based on the file position
- Use LBAread to load blocks into a buffer
- Copy only the requested number of bytes into the user buffer
- Stop at the file size to avoid returning unused data

For writing (b_write), we will implement a read-modify-write strategy:

- Read the existing block from disk
- Modify only the relevant portion of the block
- Write the entire block back using LBAwrite

This approach is necessary because the underlying storage system only supports full block writes.

We will also handle file flags carefully:

- O_CREAT will create a new file if it does not exist
- O_TRUNC will reset the file size and update metadata
- O_APPEND will move the file position to the end before writing

The b_seek function will update file position based on the specified mode, and b_close will ensure that updated file size and timestamps are written back to the directory entry.

Testing Strategy

We will test the system incrementally after each phase using the provided fsshell driver.

- In Phase 1, we will verify correct formatting and mounting behavior
- In Phase 2, we will test directory operations such as md, ls, cd, and pwd
- In Phase 3, we will test file operations such as touch, cat, cp, cp2fs, and cp2l

We will also test edge cases, such as invalid paths and empty directory deletion, to ensure robustness. Screenshots will be captured during testing to document system behavior and verify correctness.

Documentation and Writeup Plan

For the write-up, our plan was to document issues and resolutions as we worked rather than trying to remember everything at the end. Each team member was responsible for adding to a shared Google Doc whenever they ran into a problem and figured out how to fix it. That way the Issues and Resolutions section would build up naturally over the course of the project instead of being written all at once. We also planned to take screenshots of our terminal output as we tested each shell command so we would have everything ready for the screenshot sections without having to go back and redo testing at the last minute. This approach helped us stay

organized and made sure each person's contributions were captured in their own words as they worked.

Write-Up and Development Process

Our team treated documentation as an integral part of the development process rather than a final step. Throughout the project, we maintained a shared document where each team member recorded issues, design decisions, and solutions as they occurred.

Whenever a problem was encountered, we documented:

- The observed behavior
- The root cause of the issue
- The reasoning behind the chosen solution
- Any trade-offs or limitations introduced

This approach ensured that the Issues and Resolutions section accurately reflects the real challenges faced during development and provides meaningful technical insight rather than generic descriptions.

We also prioritized capturing screenshots during active testing sessions. This allowed us to document actual system behavior, including both successful command execution and failure cases. For example, the screenshots on pages 24–25 show issues encountered with the rm command, which were later analyzed and resolved.

From a collaboration standpoint, we divided responsibilities across major components (volume, directories, file I/O), but regularly reviewed each other’s code to maintain consistency. We used GitHub for version control, which introduced challenges such as merge conflicts when multiple team members modified the same files. These issues were resolved through improved communication and more structured task assignment.

Additionally, we learned the importance of testing on a clean volume. Early in development, persistent data in the SampleVolume file caused confusion when changes did not appear to take effect. To address this, we consistently deleted the volume file before testing or automated this step in the Makefile.

Overall, this process allowed us to build the system incrementally, maintain clear documentation of our work, and ensure that each team member contributed both to the implementation and understanding of the system.

Issues and Resolutions:

Issue 1: Our b_io.c was originally written calling Linux's open(), read(), and write() directly instead of going through our filesystem's LBAread/LBAwrite. This meant that file operations bypassed our directory structures entirely. Resolution pending – b_io.c needs to be fully rewritten for Phase 3.

Issue 2: The original fs_closedir stub only freed dirp->di and dirp itself, but did not free the entries array that fs_opendir loads from disk. This would cause memory leak every time a directory was closed. We resolved this by storing the entries pointer immediately after the fs_diriteminfo struct in memory and freeing it first inside fs_closedir.

Issue 3: A write can start in the middle of a block, so writing a whole block directly could overwrite data that was not supposed to change.
Resolution: Used a read-modify-write approach: read the block first, copy the new data into the correct offset, then write the block back.

Issue 4: A write request can cross multiple blocks, so one memcpy is not enough.
Resolution: Wrote the data in a loop by calculating blockIndex, offsetInBlock, and chunkSize each time until the write is finished or the allocated space is full.

Issue 5: The O_APPEND behavior can be lost in the volume path, causing data to be overwritten instead of appended. Resolution: If O_APPEND is set, move filePos to vol_file_size before performing the write.

Issue 6: There was no proper bridge between b_open and the volume layer, so even though the path worked, the actual data I/O didn't. Resolution: Added mfs_volume_open/close as a bridge. mfs_volume_open handles path parsing and finding/creating the file on the volume, and mfs_volume_close writes metadata back and frees resources. This connects the namespace with the actual data layer.

Issue 7: The original FCB was designed for host file descriptors only, so it couldn't store volume file info. Resolution: Extended the FCB in b_io to include volume-related context (like start block, block count, parent directory info, cached file size, etc.), so b_read/b_write can later work at the block level without major changes.

Issue 8: O_TRUNC should not truncate the file every time it appears in the flags. If the file is opened as read-only, it should not modify the file data. Resolution: In mfs_volume_open(), I added a condition so truncation only happens when O_TRUNC is set and the file is not opened as read-only:(flags & O_TRUNC) && ((flags & O_ACCMODE) != O_RDONLY). Only then do we set size = 0 and write the updated directory entry back.

Issue 9: Updating size only in memory is not enough. If the system crashes before close(), the truncated size could be lost. Resolution: In the truncation branch, I immediately call LBAwrite on the parent directory block, so the updated size and modifiedTime are saved to disk right away.

Issue 10: Bitmap was not loaded back into memory when mounting an existing volume. When we ran the program a second time without deleting the SampleVolume file, the system would detect that the volume was already formatted and skip the format step. The problem was that the bitmap only gets loaded into memory during formatting, so on a clean mount g_bitmap stayed NULL. Any call to allocateBlocks() would immediately fail because it checks for a NULL bitmap first. We fixed this by adding code inside fs_vol_try_mount() to read the bitmap back from disk into a new malloc'd buffer right after reading the superblock. Now the bitmap is always in memory whether the volume was just formatted or mounted from a previous session.

Issue 11: cd, rm, fs_isFile, and fs_isDir could not find files or directories created during the same session and after creating a directory with md testdir, running cd testdir would say "Could not change path." Same with rm — it would say the path is neither a file nor a directory even though ls showed it right there. The root cause was that fs_setcwd, fs_isFile, and fs_isDir all called a function called fs_vol_last_component_type which only searched the original root directory data from when the volume was first formatted. Anything created afterward wasn't visible to it. We fixed all three functions by replacing that call with ParsePath, which reads the actual current directory contents from disk and can find anything that exists at that moment.

Issue 12: FS_ROOT_DIR_BLOCKS was set to different values in mfs.c and fsInit.c Both files defined their own version of the FS_ROOT_DIR_BLOCKS constant and they got out of sync — fsInit.c was updated to 8 but mfs.c still had 4. This meant the root directory was written to disk with 8 blocks but fs_mkdir was calculating directory sizes assuming only 4 blocks. This caused mismatches in directory sizes and could lead to entries being written outside the allocated space. We fixed it by updating mfs.c to also use 8 and adding a comment explaining why 4 was not enough — at 312 bytes per entry and 512 bytes per block, 4 blocks only gives 6 slots total, which leaves just 4 usable entries after . and …

Issue 13: There was no way to free blocks back to the bitmap after deleting files or directories. We had allocateBlocks()to claim blocks from the bitmap but no matching function to give them back. This meant every time a file or directory was deleted, the directory entry was cleared but the actual disk blocks it used stayed marked as taken in the bitmap forever. Over time, this would eat up all the free space even though nothing was actually stored there. We added a freeBlocks()function that takes a starting block and a count, clears those bits in the bitmap, updates the free block count in the superblock, and writes the updated bitmap to disk so the space is actually recoverable.

Issue 14: When fs_opendir loads a directory from disk, it needs to keep that data in memory so fs_readdircan return entries one at a time across multiple calls. We needed somewhere to store the pointer to that array without adding a new field to fdDir. We solved this by allocating a slightly larger block of memory for dirp->di — big enough to hold the fs_diriteminfo struct plus one extra pointer right after it. We store the entries array pointer in that extra space and retrieve it the same way in both fs_readdir and fs_closedir. It keeps everything in one allocation and makes cleanup simple.

Issue 15: fs_readdir needs to know how many total entry slots the directory has so it knows when to stop iterating. We needed a place to store that count inside the fdDir structure that

would survive across multiple calls. Rather than adding a new field we repurposed the d_reclen field in fs_diriteminfo to hold the total entry count since we don't use it for its original purpose. We just make sure to write it back after every call to fs_readdir so it doesn't get lost.

Issue 16: When we initialize the root directory, the .. entry needs to point somewhere but root has no parent. If we left it pointing to block 0 or NULL it could cause a crash when any code tries to navigate up from root. We decided to have both . and .. in the root directory point back to root itself. That way cd .. from root just stays at root, which is the same behavior you get on a real Linux system.

Issue 17: b_read did not support volume-backed files. The original b_read only called Linux's read() system call, which requires a valid Linux file descriptor. But when a file lives inside our filesystem volume, b_open sets fd = -1 and vol_open = 1, so calling read(-1, ...) would fail silently or return garbage. This meant commands like cat, cp, and cp2l could not read any file stored on our volume. Added a static helper function b_read_volume that reads from volume-backed files using LBAread instead of read(). It uses a three-phase buffered strategy: Phase 1 drains any leftover bytes already in the internal block buffer from the previous call, Phase 2 reads full blocks directly into the caller's buffer to avoid an unnecessary memcpy, and Phase 3 reads one partial block into the internal buffer and copies only the needed bytes out, leaving the rest cached for the next call. The count is clamped to vol_file_size - filePos before any reading begins so padding bytes at the end of an allocated block are never returned as file data. At the top of b_read, a two-line check routes volume-backed files to this helper while leaving the original read()-based path completely unchanged for regular Linux files.

Issue 18: rm could not delete files or directories created in the same session fs_isFile and fs_isDir were calling fs_vol_last_component_type which only searched the original root directory data from when the volume was first formatted, so anything created afterward was invisible to it. rm would say "the path is neither a file nor a directory" even though lsshowed it right there. We fixed this by replacing the fs_vol_last_component_type call in both functions with ParsePath, which reads the actual current directory contents from disk and finds anything that exists at that moment. *(Note: this is the same root cause as Issue 11 but manifested separately in rm)*

Issue 19: fs_setcwd updated the working directory without verifying the path existed on disk. The original implementation of fs_setcwd called path_canonicalize to clean the path and then immediately stored it in g_fs_cwd without checking whether that path actually existed on disk. This meant cd into a nonexistent directory would silently succeed and leave the shell in an invalid state. We resolved this by adding a ParsePath call after canonicalization to verify the path exists and confirm its fileType is FS_FTYPE_DIR before updating g_fs_cwd.

Issue 20: fs_rmdir and fs_delete did not free bitmap blocks when deleting both functions cleared the directory entry with memset and wrote the parent back to disk, but never released the disk blocks the deleted entry had been using back to the bitmap. This meant every deletion permanently leaked space — the blocks stayed marked as used in the bitmap even though nothing was stored there. Over time, this would exhaust all free space. We resolved this by

adding a freeBlocks()function that clears the corresponding bits in g_bitmap, increments g_fs_sb.free_block_count, and flushes the updated bitmap to disk with LBAwrite.

Issue 21: Keeping struct definitions in sync across multiple files, both mfs.c and fsInit.c define their own copies of fs_superblock_t and fs_dirent_t because there was no shared private header file for internal structs. This made it easy for the two definitions to drift out of sync, as happened with FS_ROOT_DIR_BLOCKS. In a future project, we would move shared internal structs into a single internal header file that both files include, so there is only ever one definition to maintain.

Issue 22: Git merge conflicts when multiple teammates worked on the same file. Because several functions in mfs.c and fsInit.c were being developed by different team members at the same time, we occasionally ran into merge conflicts when pushing to GitHub. We resolved this by communicating more clearly about who owned which file at any given time and making sure everyone pulled the latest changes before starting new work.

Issue 23: Testing required deleting the volume file between runs during development, many bugs were only visible on a fresh volume, but the SampleVolume file would persist between runs, causing the system to mount the old volume instead of formatting a fresh one. This made it easy to accidentally test against stale data. We established a habit of always running rm -f SampleVolume before testing changes, and later added rm -f SampleVolume to the make run target in the Makefile so it happened automatically.

Issue 24: Coordinating work across four teammates remotely. Since the team worked mostly remotely over Zoom and Discord, it was sometimes difficult to coordinate who was working on what, leading to some duplicated effort early on. We resolved this by being more explicit about task assignments and checking in more frequently before starting new work to make sure we weren't stepping on each other.

Issue 25: Understanding the LBA layer before writing any code. Early on, several teammates were unsure exactly what LBAread and LBAwrite did and how blocks, block counts, and block positions related to each other. This caused some early code to pass incorrect arguments. We resolved it by carefully re-reading fsLow.h together as a team and making sure everyone understood that LBA functions always operate in whole blocks and that the position argument is a block number, not a byte offset.

Volume Control Block (Superblock) — fsInit.c

The Volume Control Block is the first structure written to disk and lives at block 0. We defined it as fs_superblock_t using __attribute__((packed)) so the compiler doesn't add padding between fields — this matters because we write the struct directly to disk with LBAwrite and read it back with LBAread, so the byte layout in memory has to match the byte layout on disk exactly.

The magic number is 0x46534331 which spells out "FSC1" in ASCII. Every time initFileSystem is called, it runs fs_vol_try_mount first, which calls read_superblock to load block 0 and checks whether sb.magic == FS_MAGIC and sb.version == FS_VERSION. If both match, the volume is already formatted and we skip formatting. If they don't match, initFileSystem calls fs_vol_format to build everything from scratch.

The superblock stores bitmap_start_lba and bitmap_block_count so we always know where the bitmap is, root_dir_start_lba and root_dir_block_count so we can find the root directory, first_data_lba which marks where actual file data begins, and free_block_count which we update every time blocks are allocated or freed. Writing the superblock goes through write_superblock, which uses calloc to allocate a full block-sized buffer, copies the struct into it with memcpy, and writes exactly one block with LBAwrite(buf, 1, 0) — we need the full block because the disk layer only accepts block-sized writes.

Free Space Management — fsInit.c

We track free space with a bitmap stored starting at block 1, immediately after the superblock. Each bit in the bitmap represents one block on the volume — 0 means free, 1 means used. For our 19,531-block volume that's 19,531 bits, which needs about 2,442 bytes, which rounds up to 5 blocks using the ceiling formula (bitmap_bytes + blockSize - 1) / blockSize.

During fs_vol_format, we call calloc to zero out the entire bitmap buffer, then call bitmap_set_used in a loop to mark blocks 0 through first_data_lba - 1 as taken since those are used by the superblock, bitmap itself, and root directory. We also mark any bits beyond total_blocks as used to prevent allocating non-existent blocks.

The bitmap pointer g_bitmap is kept as a global in memory the whole time the system is running. This means allocation and deallocation never need to read the bitmap from disk — they just update the in-memory copy and flush it back to disk after every change.

allocateBlocks(count) scans the bitmap starting from block 0 looking for count consecutive free bits. It tracks a running count of consecutive free bits using a found counter and a startBlock variable. When found == count it marks all those bits as used with g_bitmap[j/8] |= (1u << (j%8)), decrements g_fs_sb.free_block_count, flushes the bitmap with LBAwrite, and returns startBlock as an int.

freeBlocks(startBlock, count) does the reverse — it clears bits using g_bitmap[i/8] &= ~(uint8_t)(1u << (i%8)), increments free_block_count, and flushes the bitmap to disk. This is what fs_rmdir and fs_delete eventually need to call to actually reclaim space.

Directory Structure — fsInit.c and mfs.c

Directories are stored on disk as arrays of fs_dirent_t structs packed side by side. Each fs_dirent_t is defined with __attribute__((packed)) and contains a 256-byte name field, a fileType byte (FS_FTYPE_REG = 1 for files, FS_FTYPE_DIR = 2 for directories), an inUse byte, startBlock and blockCount as uint64_t, size as uint64_t, and three time_t timestamps for createTime, modifiedTime, and accessTime.

Every directory is allocated exactly FS_ROOT_DIR_BLOCKS = 8 blocks, giving 8 * 512 = 4096 bytes per directory. Since each fs_dirent_t is 312 bytes, that fits 4096 / 312 = 13 entries per directory, with slots 0 and 1 always reserved for . and ... We increased this from 4 to 8 blocks because 4 blocks only gave 6 total slots and just 4 usable ones, which wasn't enough for practical use.

When the volume is first formatted, initRootDir uses calloc(FS_ROOT_DIR_BLOCKS, blockSize) to zero out the root directory buffer. Slot 0 gets the . entry pointing to root_dir_start with blockCount = FS_ROOT_DIR_BLOCKS and size = dirBytes. Slot 1 gets ..pointing to the same location since root has no parent — both . and .. point back to root itself. The whole buffer is written in one LBAwrite(root, FS_ROOT_DIR_BLOCKS, root_dir_start) call.

Path Resolution — mfs.c

All path operations go through two functions: path_canonicalize and ParsePath.

path_canonicalize takes the current working directory string, the path the user typed, and an output buffer. It first builds a raw combined path — if the input starts with / it's already absolute, if the cwd is / it prepends just /, otherwise it prepends cwd/. It then makes a copy with strncpy because strtok_r modifies the string in place. It walks each token separated by / — skipping, popping the last element off a parts[] array for .., and pushing everything else. At the end it reassembles the surviving parts into the output buffer with a leading / separator. This correctly handles paths like ../../foo from /a/b/cproducing /a/foo.

resolve_lookup_path is a thin wrapper around path_canonicalize that always passes g_fs_cwd as the current directory, so every other function in mfs.c can call it with just the user path and get a clean absolute path back.

ParsePath takes that clean absolute path and walks the actual directory tree on disk. It builds a temporary fs_dirent_t for the root using g_fs_sb.root_dir_start_lba and g_fs_sb.root_dir_block_count from the global superblock, then calls LoadDir to read the root from disk into a malloc'd buffer. For a path like /testdir/subdir, it uses strtok_r to split into tokens and processes them in pairs — looking ahead with a next pointer to know whether the current token is a middle component (must exist and be a directory) or the last component (fills in ppi.index and ppi.lastElementName). For middle components it calls FindEntryInDir to find the slot, calls LoadDir on that entry, frees the current directory buffer, and descends. The special

index value -2 signals that the path was / and the root itself is the target. The caller always receives ppi.parent (loaded directory) and ppi.lastElementName (malloc'd string) and is responsible for freeing both.

FindEntryInDir computes the entry count by dividing dir[0].size by sizeof(fs_dirent_t), then scans slots where inUse == 1 looking for a strcmp match on the name.

Directory Operations — mfs.c

fs_mkdir calls ParsePath to get the parent directory and check whether the name already exists (ppi.index >= 0 means duplicate). It calls allocateBlocks(FS_ROOT_DIR_BLOCKS) to get 8 blocks for the new directory, uses calloc to zero a buffer, fills in slot 0 as . pointing to the new blocks and slot 1 as .. pointing to ppi.parent[0].startBlock and ppi.parent[0].blockCount, and writes it to disk. Then it scans the parent's slots for one where inUse == 0, fills it in with the new entry's metadata, and calls LBAwrite on the parent to save the change permanently.

fs_rmdir calls ParsePath to find the target, verifies fileType == FS_FTYPE_DIR, calls LoadDir on the target to check whether it has any entries besides . and .. by scanning slots where inUse == 1 and the name is not . or ... If any such slot exists it returns ENOTEMPTY. Otherwise, it clears the parent's slot with memset and writes the parent back to disk.

fs_opendir calls ParsePath, handles the three index cases (-2 for root, >= 0 for found entry, -1 for not found), reads the directory's blocks into a malloc'd entries array using LBAread, allocates an fdDir struct, and allocates dirp->di as sizeof(struct fs_diriteminfo) + sizeof(fs_dirent_t*) — one extra pointer's worth of space right after the fs_diriteminfo struct. The total entry count is stored in dirp->di->d_reclen and the entries pointer is stored at (fs_dirent_t**)(dirp->di + 1).

fs_readdir recovers both by reading dirp->di->d_reclen for the count and dereferencing (fs_dirent_t**)(dirp->di + 1) for the entries array. It advances current_index past any inUse == 0 slots, copies the name and fileType into dirp->di, restores d_reclen so it isn't lost, and returns dirp->di. Returns NULL when current_index >= nEntries.

fs_closedir frees the entries array by recovering the pointer from (fs_dirent_t**)(dirp->di + 1), then frees dirp->di and dirp itself.

Working Directory — mfs.c

The current working directory is stored as the global char g_fs_cwd[FS_CWD_MAX] initialized to "/". FS_CWD_MAX is 4096, which matches the Linux PATH_MAX.

fs_getcwd just copies g_fs_cwd into the caller's buffer with memcpy after checking that strlen(g_fs_cwd) + 1 fits in the given size.

fs_setcwd first calls path_canonicalize to get a clean absolute path into canon[], then calls ParsePath(canon, &ppi) to verify the path actually exists on disk. If ppi.index != -2 (not root) it

checks ppi.parent[ppi.index].fileType == FS_FTYPE_DIR to confirm it's a directory. After freeing ppi.parent and ppi.lastElementName it copies canon into g_fs_cwd with memcpy(g_fs_cwd, canon, strlen(canon) + 1).

fs_isFile and fs_isDir both call ParsePath and check ppi.parent[ppi.index].fileType against FS_FTYPE_REG or FS_FTYPE_DIRrespectively. fs_isDir also handles ppi.index == -2 which is always true for /.

File I/O — b_io.c

The FCB table is b_fcb fcbArray[MAXFCBS] where MAXFCBS = 20. Each b_fcb has a char* buf for the block buffer, int index and int buflen for the buffer cursor, int fd for a Linux file descriptor (or -1 for volume files), int filePos and int fileSize, int flags, and a vol_open flag plus an mfs_b_open_ctx vol struct and uint64_t vol_file_size for volume-backed files.

b_init zeroes all buf pointers so b_getFCB can find a free slot by scanning for fcbArray[i].buf == NULL.

b_open calls mfs_volume_open first. If it returns 0 the file is on our volume — it sets vol_open = 1, fd = -1, allocates a block-sized buffer using ctx.fs_block_size, and stores all the volume context. If mfs_volume_open fails, it falls back to the Linux open() call for files that live on the host filesystem.

mfs_volume_open in mfs.c calls ParsePath to find the file. If found and O_TRUNC is set and the file isn't read-only (checked with (flags & O_ACCMODE) != O_RDONLY), it sets size = 0 and immediately calls LBAwrite on the parent directory to save the truncation to disk before returning. If O_CREAT is set and the file doesn't exist, it calls allocateBlocks(1) to get one block, fills in a new fs_dirent_t slot in the parent, and writes the parent to disk. The ctx struct gets filled with file_start_lba, file_block_count, file_size, and fs_block_size so b_read and b_write can access the data without going through the directory again.

b_read_volume reads data one block at a time in a loop. Each iteration computes blk = filePos / bs and off = filePos % bs, calls LBAread(f->buf, 1, file_start_lba + blk) to load that block, and copies only the bytes the caller asked for using memcpy(buffer + copied, f->buf + off, chunk). It stops when filePos >= vol_file_size so it never returns uninitialized block data.

b_write_volume also loops per block. Each iteration reads the block first with LBAread, copies new data into f->buf + off with memcpy, writes it back with LBAwrite, then advances filePos and updates vol_file_size if the write extends the file. This read-modify-write approach is necessary because LBAwrite always writes full blocks — you can't write just part of a block without reading what was already there first.

b_seek computes the new position based on SEEK_SET, SEEK_CUR, or SEEK_END, clamps it to the allocated capacity for volume files using vol_capacity_bytes(f) which returns file_block_count * fs_block_size, and calls vol_reset_buf_cursor to clear the buffer so stale cached data isn't returned on the next read.

b_close calls mfs_volume_close which writes the final vol_file_size back to dir[entry].size and updates dir[entry].modifiedTime with the current time, then calls LBAwrite on the parent directory to persist the size change. This is what makes file sizes correct after writing — without this the directory entry would still show size 0.

Table of Who Worked on What

Function / Component	Team Member(s)
fs_superblock_t struct definition	Leslie Raya
initFileSystem / exitFileSystem	Jiawei Lyu
fs_vol_try_mount	Jiawei Lyu
fs_vol_format	Leslie Raya
initRootDir	Jiawei Lyu
allocateBlocks	Leslie Raya
freeBlocks	Leslie Raya
bitmap_set_used	Leslie Raya
path_canonicalize	Alex Borders
resolve_lookup_path	Alex Borders
LoadDir	Alex Borders
FindEntryInDir	Alex Borders

ParsePath	Alex Borders, Jiawei Lyu, Yeraldin
fs_mkdir	Leslie Raya, Alex Borders, Jiawei Lyu
fs_rmdir	Leslie Raya, Alex Borders, Jiawei Lyu, Yeraldin
fs_opendir	Alex Borders
fs_readdir	Alex Borders
fs_closedir	Alex Borders
fs_getcwd	Alex Borders, Jiawei Lyu
fs_setcwd	Alex Borders, Yeraldin, Jiawei Lyu
fs_isFile	Jiawei Lyu
fs_isDir	Jiawei Lyu
fs_delete	Jiawei Lyu
fs_stat	Leslie Raya
mfs_volume_open	Jiawei Lyu
mfs_volume_close	Jiawei Lyu
b_open	Alex Borders, Jiawei Lyu

b_read	Alex Borders
b_write	Jiawei Lyu
b_seek	Alex Borders, Jiawei Lyu
b_close	Alex Borders, Jiawei Lyu, Leslie Raya
fsshell.c	Alex Borders, Jiawei Lyu, Leslie Raya, Yeradin
driver / testing	Alex Borders
Writeup / documentation	Alex Borders, Jiawei Lyu, Yeraldin

Table of Shell Commands — Status and Limitations

Command	Status	Notes / Limitations
`ls`	Works	Lists all files and directories in the current directory. Does not support sorting.
`ls -a`	Works	Shows hidden entries including `.` and `..`
`ls -l`	Partial	Runs without crashing but long format size display may not be accurate for all files

`cd`	Works	Supports relative paths, absolute paths, and `..` navigation
`md`	Works	Creates new directories with `.` and `..` entries initialized correctly
`pwd`	Works	Always returns correct absolute path, including after nested `cd` operations
`touch`	Works	Creates an empty file in the current directory
`cat`	Works	Displays file contents correctly. Limited to reading — cannot write interactively
`rm`	Works	Removes regular files with `fs_delete` and empty directories with `fs_rmdir`. Will refuse to remove non-empty directories
`cp`	Works	Copies a file within the filesystem. Source and destination must be in the same directory
`mv`	Works	Shell command moves/renames files within the filesystem correctly
`cp2fs`	Works	Copies a file from the Linux host filesystem into our filesystem correctly
`cp2l`	Works	Copies a file from our filesystem out to the Linux host filesystem correctly
`history`	Works	Displays all commands entered in the current session
`help`	Works	Lists all available commands with descriptions

Analysis:

This project successfully demonstrates the design and implementation of a persistent, block-based file system using a Logical Block Addressing interface. The system supports core file system functionality, including directory navigation, file creation, file reading and writing, and data transfer between the custom file system and the host operating system.

One of the most significant challenges in this project was bridging the gap between high-level file operations and low-level block-based storage. Because all reads and writes must occur in fixed-size blocks, special care was required to correctly handle partial block operations. This was addressed through a read-modify-write strategy, ensuring that existing data within a block is preserved when only a portion of that block is updated.

The bitmap-based free space manager proved to be an efficient solution for tracking allocation and deallocation of disk blocks. However, it required careful synchronization with disk writes to ensure that block usage remained consistent across program executions. Similarly, maintaining accurate metadata—such as file sizes and timestamps—required explicit updates to directory entries and immediate persistence in some cases, such as file truncation.

Testing through the shell interface confirmed that all commands function correctly, including ls, cd, md, touch, cat, cp, mv, cp2fs, and cp2l. While some limitations remain—such as earlier issues with rm—the system demonstrates correct behavior for all required operations.

Overall, this project provided a deeper understanding of file system design, particularly in areas such as block-level storage management, metadata consistency, and the relationship between directory structures and data access. It highlights the complexity involved in building even a simplified file system and reinforces the importance of careful design, testing, and debugging.

Since this project runs in user space rather than inside the real Linux kernel, our implementation uses memcpy() to move data between the caller's buffer and the file system's internal block buffer. In a real kernel-level file system or device driver, this transfer would need to use special kernel helper functions instead. copy_from_user() would be used when copying data from user memory into kernel memory, such as during a write operation. copy_to_user() would be used when copying data from kernel memory back to user memory, such as during a read operation. These functions are necessary in kernel code because user-space memory cannot be accessed directly and must be checked safely.

Screen shot of compilation:

```
student@student:~/csc415-group-project$ make clean
rm fsshell.o fsInit.o b_io.o mfs.o fsshell
student@student:~/csc415-group-project$ make
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -o fsshell fsshell.o fsInit.o b_io.o mfs.o fsLowM1.o -g -I. -lm -l readline -l pthread
```

Screen shot(s) of the execution of the program:

```
student@student:~/csc415-group-project$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|---------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|---------------------------------|
Prompt >
```

Commands: pwd and ls (below)

```
student@student:~/csc415-group-project$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|--------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|--------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt >
```

Commands: md testdir and ls (below)

```
student@student:~/csc415-group-project$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|--------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|--------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt > md testdir
Prompt > ls

testdir
Prompt >
```

Commands: cd testdir, pwd, cd .., pwd (below)

```
student@student:~/csc415-group-project$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|--------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|--------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt > md testdir
Prompt > ls

testdir
Prompt > md testdir
Prompt > pwd
/
Prompt > cd ..
Prompt > pwd
/
Prompt >
```

Commands: touch testfile.txt and ls (below)

```
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|--------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|--------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt > md testdir
Prompt > ls

testdir
Prompt > md testdir
Prompt > pwd
/
Prompt > cd ..
Prompt > pwd
/
Prompt > touch testfile.txt
Prompt > ls

testdir
testfile.txt
Prompt >
```

Commands: cp2fs /etc/hostname myhost.txt and cat myhost.txt (below)

```
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|---------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|---------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt > md testdir
Prompt > ls

testdir
Prompt > md testdir
Prompt > pwd
/
Prompt > cd ..
Prompt > pwd
/
Prompt > touch testfile.txt
Prompt > ls

testdir
testfile.txt
Prompt > cp2fs /etc/hostname myhost.txt
Prompt > cat myhost.txt
student
Prompt >
```

Commands: cp myhost.txt myhost2.txt and ls (below)

```
student@student: ~/CSC415/csc415-group-project — make run
~/CSC415/csc415-group-project

| pwd                    |    ON    |
| touch                  |    ON    |
| cat                    |    ON    |
| rm                     |    ON    |
| cp                     |    ON    |
| mv                     |    ON    |
| cp2fs                  |    ON    |
| cp2l                   |    ON    |
|--------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt > md testdir
Prompt > ls

testdir
Prompt > cd testdir
Prompt > pwd
/testdir
Prompt > cd ..
Prompt > pwd
/
Prompt > touch testfile.txt
Prompt > ls

testdir
testfile.txt
Prompt > cp2fs /etc/hostname myhost.txt
Prompt > cat myhost.txt
student
Prompt > cp myhost.txt myhost2.txt
Prompt > ls

testdir
testfile.txt
myhost.txt
myhost2.txt
Prompt >
```

Commands: cp2fs /etc/hostname myhost.txt and cp2l myhost.txt /tmp/myhost_copy.txt (below)

```
student@student: ~/CSC415/csc415-group-project — make run
~/CSC415/csc415-group-project

| cat                    |    ON    |
| rm                     |    ON    |
| cp                     |    ON    |
| mv                     |    ON    |
| cp2fs                  |    ON    |
| cp2l                   |    ON    |
|--------------------------------|
Prompt > pwd
/
Prompt > ls

Prompt > md testdir
Prompt > ls

testdir
Prompt > cd testdir
Prompt > pwd
/testdir
Prompt > cd ..
Prompt > pwd
/
Prompt > touch testfile.txt
Prompt > ls

testdir
testfile.txt
Prompt > cp2fs /etc/hostname myhost.txt
Prompt > cat myhost.txt
student
Prompt > cp myhost.txt myhost2.txt
Prompt > ls

testdir
testfile.txt
myhost.txt
myhost2.txt
Prompt > cp2fs /etc/hostname myhost.txt
Prompt > cp2l myhost.txt /tmp/myhost_copy.txt
Prompt >
```

Commands: rm myhost2.txt, ls, md emptydir, rm emptydir, ls

```
student@student: ~/CSC415/csc415-group-project — make run
~/CSC415/csc415-group-project

testdir
Prompt > cd testdir
Prompt > pwd
/testdir
Prompt > cd ..
Prompt > pwd
/
Prompt > touch testfile.txt
Prompt > ls

testdir
testfile.txt
Prompt > cp2fs /etc/hostname myhost.txt
Prompt > cat myhost.txt
student
Prompt > cp myhost.txt myhost2.txt
Prompt > ls

testdir
testfile.txt
myhost.txt
myhost2.txt
Prompt > cp2fs /etc/hostname myhost.txt
Prompt > cp2l myhost.txt /tmp/myhost_copy.txt
Prompt > rm myhost2.txt
Prompt > ls

testdir
testfile.txt
myhost.txt
Prompt > md emptydir
Prompt > rm emptydir
Prompt > ls

testdir
testfile.txt
myhost.txt
Prompt >
```

Commands: history and help (below)

```
student@student: ~/CSC415/csc415-group-project — make run
~/CSC415/csc415-group-project

myhost.txt
Prompt > history
pwd
ls
md testdir
ls
cd testdir
pwd
cd ..
pwd
touch testfile.txt
ls
cp2fs /etc/hostname myhost.txt
cat myhost.txt
cp myhost.txt myhost2.txt
ls
cp2fs /etc/hostname myhost.txt
cp2l myhost.txt /tmp/myhost_copy.txt
rm myhost2.txt
ls
md emptydir
rm emptydir
ls
history
Prompt > help
ls      Lists the file in a directory
cp      Copies a file - source [dest]
mv      Moves a file - source dest
md      Make a new directory
rm      Removes a file or directory
touch   Touches/Creates a file
cat     Limited version of cat that displace the file to the console
cp2l    Copies a file from the test file system to the linux file system
cp2fs   Copies a file from the Linux file system to the test file system
cd      Changes directory
pwd     Prints the working directory
history Prints out the history
help    Prints out help
Prompt >
```

Commands: (Nested commands) md testdir2, cd testdir2, md subdir, ls, cd subdir, pwd, cd ../.., pwd (below)

student@student: ~/CSC415/csc415-group-project — make run
~/CSC415/csc415-group-project

```
cp2fs /etc/hostname myhost.txt
cat myhost.txt
cp myhost.txt myhost2.txt
ls
cp2fs /etc/hostname myhost.txt
cp2l myhost.txt /tmp/myhost_copy.txt
rm myhost2.txt
ls
md emptydir
rm emptydir
ls
history
Prompt > help
ls      Lists the file in a directory
cp      Copies a file - source [dest]
mv      Moves a file - source dest
md      Make a new directory
rm      Removes a file or directory
touch   Touches/Creates a file
cat     Limited version of cat that displace the file to the console
cp2l    Copies a file from the test file system to the linux file system
cp2fs   Copies a file from the Linux file system to the test file system
cd      Changes directory
pwd     Prints the working directory
history Prints out the history
help    Prints out help
Prompt > md testdir
Prompt > cd testdir
Prompt > md subdir
Prompt > ls

subdir
Prompt > cd subdir
Prompt > pwd
/testdir/subdir
Prompt > cd ../..
Prompt > pwd
/
Prompt >
```

Command: make run → ls and pwd (below)

```
student@student:~/csc415-group-project$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
File system ready.
|--------------------------------|
|------- Command ------|- Status -|
| ls                  |    ON    |
| cd                  |    ON    |
| md                  |    ON    |
| pwd                 |    ON    |
| touch               |    ON    |
| cat                 |    ON    |
| rm                  |    ON    |
| cp                  |    ON    |
| mv                  |    ON    |
| cp2fs               |    ON    |
| cp2l                |    ON    |
|--------------------------------|
Prompt > ls

testdir
testfile.txt
myhost.txt
myhost2.txt
Prompt > pwd
/
Prompt >
```